

AI-POWERED EDUCATIONAL CHATBOT

EduBot AI

Name: RAHUL RAJ

DESIGNATION: AI/ML INTERN

COMPANY NAME: APANA TIME TECH SOLUTIONS

DATE: 15TH AUGUST TO 8TH SEPTEMBER

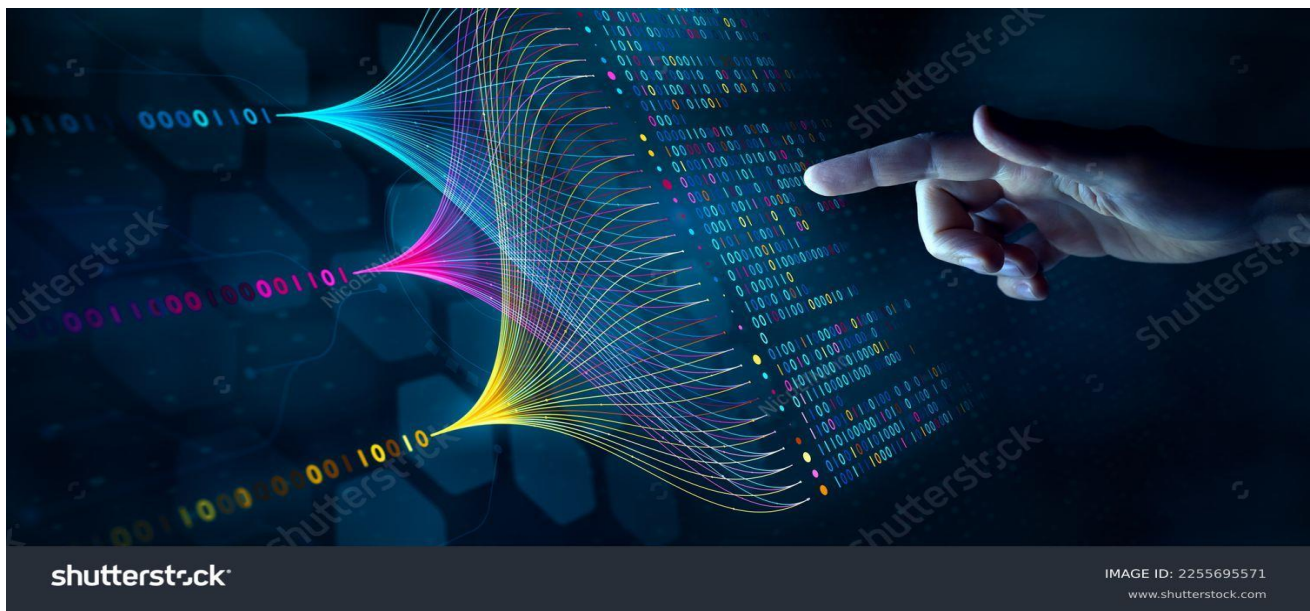


Table of Contents:

TOPICS	PAGES
1. Abstract	3-4
2. Introduction	4-5
3. Problem Statement	5
4. Objectives	5-6
5. Scope of the Project	6
6. System Requirements	7
7. Dataset / CSV Knowledge Base	7-8
8. Data Cleaning and Preprocessing	8
9. Sentence Transformer	8-9
10. Text Embedding	9
11. Cosine Similarity	9-10

12. Similarity Threshold and Fallback	10
13. Context-Aware Follow-Up Question	10-11
14. System Architecture	11-12
15. Stream lit User Interface	12
16. Session-Based Chat History	13
17. SQLite Persistent Chat History	13-14
18. Complete System Workflow	14
19. Testing and Validation	14-15
20. Results and Discussion	15
21. Advantage	16
22. Limitation	16
23. Future Scope	16-17
24. References	17-18

1. Abstract

EduBot AI is an AI-powered educational question-answering chatbot designed to provide fast and relevant answers to questions contained in a structured educational (**Data Science**) knowledge base. The project uses a CSV file containing approximately question-answer pairs as its primary source of knowledge in **Data Science** field. Instead of depending on simple keyword matching, the system uses a pretrained Sentence Transformer model, all-MiniLM-L6-v2, to represent questions as semantic numerical embeddings. When a user submits a question, the same model converts the query into an embedding and the system compares it with the stored question embeddings using cosine similarity. The question with the highest similarity score is selected as the candidate match. A similarity threshold is used to decide whether the candidate is sufficiently relevant. If the score meets the configured threshold, EduBot returns the corresponding answer from the CSV file. If the score is below the threshold, the chatbot produces a controlled fallback message rather than returning a potentially irrelevant answer. This makes the retrieval system more reliable and transparent. A key functionality is context-aware follow-up handling. The chatbot can retain the current conversational topic so that short follow-up questions can be interpreted meaningfully. For a genuinely new subject, the system can switch the active topic and retrieve from the new context. The application is implemented using Streamlit, which provides the interactive web interface. The interface includes a chat input, suggested questions, a similarity-threshold control, context-aware follow-up handling, retrieval details, and a clear-history option. Session state is used to maintain the conversation during an active Streamlit session. In addition, SQLite3 provides persistent storage for question-answer conversations, allowing users to view previous chats even after refreshing or restarting the application. Overall, EduBot AI demonstrates a practical retrieval-based NLP system that combines semantic search, a curated educational (**Data Science**) dataset, an interactive interface, and persistent chat storage. The project can serve as a foundation for future improvements such as larger knowledge bases, multilingual support, document retrieval, user authentication, hybrid search, and integration with generative AI.

2. Introduction

Artificial Intelligence is increasingly being used to improve the way students access information and learning resources. Educational institutions and online learning platforms commonly contain large amounts of information, but students may still spend significant time searching for straightforward explanations. A conversational interface can reduce this effort by allowing a learner to ask a question in natural language and receive a relevant response.

EduBot AI was developed as a focused educational chatbot that retrieves answers from a predefined knowledge base of **Data Science** field. The central idea is to make the interaction simple: a student enters a question, the application identifies the most semantically similar question in its dataset, and the associated answer is displayed. The system therefore combines natural-language processing with information retrieval rather than generating unrestricted answers.

The knowledge base of **Data Science** is maintained in CSV format, with separate question and answer columns. This makes the system easy to update because new educational question-answer pairs can be added without redesigning the application. Before retrieval, the dataset is cleaned by converting text fields to strings, trimming unnecessary spaces, normalizing repeated whitespace, removing duplicate pairs, and removing empty records.

The retrieval layer uses the **all-MiniLM-L6-v2** Sentence Transformer. This model converts questions into embeddings that capture semantic information. **Cosine similarity** then measures how closely the user's query corresponds to each stored question. A threshold determines whether the best match is acceptable.

In this workflow, the **context memory** component in EduBot AI is responsible for understanding the relationship between the user's current question and the questions asked earlier in the conversation.

Instead of treating every question as completely independent, the chatbot maintains an **active topic/context** from the previous conversation and uses it to interpret follow-up questions. The front end is implemented in Streamlit and includes a modern chat experience, suggested questions, a knowledge-based counter, a configurable similarity threshold, follow-up context handling memory, retrieval details, and persistent chat history. Together, these components create a lightweight educational assistant that is easy to understand, test, maintain, and demonstrate.

3. Problem Statement

Students frequently need quick explanations of concepts of **Data Science** such as Artificial Intelligence, Machine Learning, Python, Statistics, Natural Language Processing, Computer Vision, SQL, and Generative AI. In a traditional learning environment, students may search through notes, websites, PDFs, or frequently asked questions to locate an answer. This process can be time-consuming, especially when the student knows the concept but phrases the question differently from the wording used in the available material.

A basic keyword-based chatbot may also have difficulty handling semantic variations and follow-up current related question. For example, a knowledge base might contain the question “What is Artificial Intelligence?” while a student asks “Can you explain AI?” A purely literal search may treat these as different strings even though their meanings are closely related. Therefore, the project requires a retrieval approach that considers semantic similarity rather than only exact word overlap. Example of follow up question like if the user first asks “What is Python?” the chatbot identifies Python as the active topic. If the user then asks “What is its type?” this question by itself is ambiguous because the word “*its*” does not specify what it refers to. The context memory connects it with the previous topic and internally interprets the question as something like “What are the types of Python data types?” allowing the semantic-search system to retrieve a more relevant answer. Similarly, if the user changes the topic by asking “What is SQL?” the chatbot updates the active topic from Python to SQL. If the next question is “What is its type?” the chatbot now understands that “*its*” refers to SQL and searches for information related to SQL data types. This mechanism combines conversation history, active-topic detection, follow-up-question detection, and semantic search.

Another problem is answer reliability. If a chatbot always returns the closest record, it may produce an answer even when the user's question is unrelated to the educational dataset. EduBot addresses this through a similarity threshold. A candidate answer is returned only when its similarity score reaches the configured threshold. Otherwise, a fallback response tells the user that a relevant answer could not be found in the knowledge base.

The project also requires an accessible user interface and basic conversation management. Users should be able to ask questions naturally, see previous messages, use suggested questions, inspect retrieval information, and clear the conversation. For a more complete application, chat history should survive browser refreshes and application restarts. This led to the addition of SQLite persistent storage.

The problem addressed by EduBot AI is therefore the design of a lightweight educational question-answering system that can perform semantic retrieval, reject weak matches, provide a usable web interface, and maintain persistent chat history without requiring a large generative language model or an external API.

4. Objectives

The primary objective of EduBot AI is to build an educational chatbot that can retrieve relevant answers from a structured question-answer dataset using semantic similarity. The project is designed to demonstrate the complete workflow from data preparation to user interaction.

The first objective is to create and maintain a structured CSV knowledge base containing approximately educational question-answer pairs. The dataset should cover the intended educational (**Data Science**) topics and provide clear answers that can be returned directly to users.

The second objective is to perform basic text cleaning. Questions and answers are converted to strings, leading and trailing spaces are removed, repeated whitespace is normalized, duplicate question-answer pairs are removed, and empty records are excluded.

The third objective is to implement semantic retrieval using a Sentence Transformer. The all-MiniLM-L6-v2 model is used to encode stored questions and incoming user queries into numerical embeddings. These representations allow the system to compare meaning rather than relying only on exact text matching.

The fourth objective is to calculate cosine similarity and select the highest-scoring candidate. A configurable similarity threshold is then applied so that weak or unrelated queries can receive a fallback response.

The fifth objective is to handle follow-up questions using the active conversation topic and switch the active topic when a genuinely new topic is detected.

The sixth objective is to develop an interactive Streamlit interface. The interface supports normal chat input, suggested questions, a similarity-threshold slider, follow-up context handling, retrieval details, and a clear-history option.

The last objective is to maintain conversation state using Streamlit session state and persistent storage using SQLite. The final system saves question-answer pairs, similarity scores, follow-up-question, matched questions, match status, and timestamps. Finally, the project aims to provide a clear, testable, documented system suitable for demonstration, academic submission, and future extension.

5. Scope of the Project

The scope of EduBot AI is focused on educational question answering from a curated knowledge base of **Data Science**. The application is intended to assist users with common educational questions in areas represented in the CSV dataset, including Artificial Intelligence, Machine Learning, Deep Learning, Python, Statistics, NLP, Computer Vision, SQL, and Generative AI.

The project covers the complete retrieval pipeline. It begins with a CSV knowledge base, performs basic cleaning, creates semantic embeddings for stored questions, accepts a natural-language user query, generates a query embedding, calculates cosine similarity, selects the best match, checks a threshold, provide answer based on follow-up-context question either the matched answer or a fallback response.

The user interface is also within the project scope. Streamlit provides the web application, including the chat interface, sidebar, suggested questions, knowledge-based information, retrieval details, and clear-history control. SQLite stores conversations locally. The user can refresh or restart the application and still retrieve previously stored conversation records.

The project is not designed as a general-purpose open-domain AI assistant. It does not independently know information outside its knowledge base, and it does not generate original answers using a large language model. Consequently, its accuracy and coverage depend on the quality, clarity, and breadth of the CSV dataset. The similarity threshold also influences how strict the system is when deciding whether a question is relevant.

Possible deployment scope includes classroom support tools, small educational portals, training applications, internal FAQ systems, and prototype learning assistants. For larger production environments, additional capabilities such as authentication, multi-user database design, cloud storage, logging, monitoring, stronger retrieval, and potentially RAG or LLM integration would be required.

6. System Requirements

EduBot AI requires a standard development computer capable of running Python, Streamlit, Pandas, NumPy, scikit-learn, and the Sentence Transformers model. A modern multi-core processor is useful for installing dependencies and generating embeddings. At least 8 GB of RAM is recommended for a comfortable development experience, although the exact requirement depends on the operating system and model environment. Adequate storage is required for Python packages, the Sentence Transformer model, the dataset, and project files.

The software environment includes Python as the main programming language. Pandas is used for loading and cleaning the CSV dataset. NumPy supports numerical operations such as selecting the maximum similarity index. Sentence Transformers provides the pretrained semantic embedding model. Scikit-learn provides the cosine similarity implementation. Streamlit is used to create the web application and chat interface. SQLite is used as the local persistent database, and the sqlite3 module is included with standard Python installations.

A suitable development environment may include VS Code, PyCharm, or another Python-compatible IDE. A web browser such as Chrome, Edge, or Firefox is required to access the Streamlit application. `pip install streamlit pandas numpy sentence-transformers scikit-learn`

Category	Requirement	Purpose
Language	Python	Application development
Data	Pandas	CSV loading and cleaning
Numerical	NumPy	Numerical operations
NLP	Sentence Transformers	Semantic embeddings
Similarity	Scikit-learn	Cosine similarity
UI	Streamlit	Web interface
Database	SQLite / sqlite3	Persistent chat history

7. Dataset / CSV Knowledge Base

The knowledge base is the central source of information of (**Data Science**) for EduBot AI. It is stored in a CSV file named `knowledge_datascience.csv` and contains question-answer pairs. The project uses two required columns from the file: question and answer. Each question represents a concept or information request that a user may submit, while the corresponding answer is the response returned when that question is selected as the best semantic match.

The CSV structure is intentionally simple. A typical record can be represented as “What is Machine Learning?” in the question column and an explanatory definition in the answer column. This structure makes the dataset easy to inspect using spreadsheet software, Pandas, or a text editor. It also makes adding new questions straightforward.

During loading, the application selects only the required columns. The text fields are converted to strings to avoid type inconsistencies. Leading and trailing spaces are removed, and repeated whitespace is normalized. Duplicate question-answer pairs are removed so that redundant records do not unnecessarily influence retrieval. Empty questions and answers are excluded.

The quality of the knowledge base strongly influences the chatbot's usefulness. Questions should be clearly worded, answers should be concise but informative, and different ways of asking the same concept can be represented through multiple question records if desired. The dataset should also maintain consistent educational terminology.

The chatbot is retrieval-based: it does not create new answers from the internet. Instead, it finds a relevant question in this curated knowledge base and returns its stored answer. This makes the system predictable and easier to validate.

question	answer
What is Artificial Intelligence?	Stored educational explanation of Artificial Intelligence.
What is Machine Learning?	Stored educational explanation of Machine Learning.
What is NLP?	Stored educational explanation of Natural Language Processing.

8. Data Cleaning and Preprocessing

Data cleaning prepares the CSV knowledge base for reliable semantic encoding. EduBot performs lightweight preprocessing because Sentence Transformers are designed to work with natural-language sentences. The goal is to remove obvious formatting problems without aggressively altering the wording.

First, the question and answer columns are converted to strings using as type (str). This ensures that all records are represented consistently before string operations are performed. Leading and trailing whitespace is then removed using str.strip(). Repeated whitespace is replaced with a single space using a regular-expression replacement. These steps reduce accidental formatting variation.

The project also removes duplicate question-answer pairs with drop_duplicates. Duplicate records can create unnecessary redundancy and may make the retrieval result less informative. Empty questions and answers are removed because they cannot provide useful retrieval or response content.

The project does not require extensive traditional NLP preprocessing such as stemming or lemmatization before Sentence Transformer encoding. Stemming can transform words into shortened forms, and aggressive preprocessing can sometimes remove contextual information that semantic models can use. The Sentence Transformer can process natural language directly, so the project focuses on clean, readable sentences.

This preprocessing stage creates a consistent and compact knowledge base before embeddings are generated.

```
df["question"] = df["question"].astype(str)
df["answer"] = df["answer"].astype(str)
df["question"] = df["question"].str.strip()
df["answer"] = df["answer"].str.strip()
df["question"] = df["question"].str.replace(r"\s+", " ", regex=True)
df["answer"] = df["answer"].str.replace(r"\s+", " ", regex=True)
df = df.drop_duplicates(subset=["question", "answer"]).reset_index(drop=True)
```

9. Sentence Transformer

Sentence Transformers are neural-network-based models designed to convert sentences and short passages into dense numerical vectors called embeddings. These embeddings are useful for semantic search because texts with related meanings can be represented by vectors that are closer to one another in the embedding space.

EduBot uses the pretrained model all-MiniLM-L6-v2. The model is loaded through the sentence-transformers library. When the application starts, it encodes all questions in the CSV knowledge base. The resulting vectors are kept in memory for retrieval. Because the questions do not change during a session, this avoids recomputing the knowledge-based embeddings for every user query.

The model is useful because the chatbot should understand variations in wording. A user may ask “What is AI?” while the knowledge base contains “What is Artificial Intelligence?” The two questions are not identical strings, but their meanings are closely related. A semantic embedding model can capture this relationship more effectively than simple exact matching.

The application uses `normalize_embeddings=True` during encoding. Normalization produces unit-length vectors, which makes cosine-based comparisons convenient and consistent.

For a project report, this section should emphasize that the model is not being trained from scratch. It is a pretrained model being used as an embedding engine for semantic retrieval.

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
question_embeddings = model.encode(
    df["question"].tolist(),
    normalize_embeddings=True,
    show_progress_bar=False
)
```

10. Text Embedding

Text embedding is the process of converting natural-language text into a numerical vector that represents semantic information. In EduBot, embeddings form the bridge between the user's natural-language question and the structured CSV knowledge base. Each stored question is converted into an embedding when the application initializes, and each new user query is converted into another embedding at runtime.

The knowledge-base embeddings are created using the Sentence Transformer model. The application sends the complete list of cleaned questions to `model.encode`. The resulting numerical matrix contains one vector for each question. These vectors are stored in memory and reused for future queries.

When a user asks a question, the application calls `model.encode` for that single query. The query vector is then compared with the matrix of stored question vectors. This design is efficient for a small knowledge base because the knowledge-base encoding is performed once and cached using Streamlit's resource caching.

Embedding allows the system to move from raw text comparison to numerical semantic comparison. Instead of asking whether two strings contain the same words, the retrieval system asks which stored question has the closest vector representation to the user's query.

```
question_embeddings = model.encode(
    df["question"].tolist(),
    normalize_embeddings=True,
    show_progress_bar=False
)
query_embedding = model.encode(
    user_question,
    normalize_embeddings=True
)
```

11. Cosine Similarity

Cosine similarity is the main retrieval measure used by EduBot AI. It compares two vectors by measuring the cosine of the angle between them. In semantic search, a higher cosine similarity generally indicates that two embeddings are more closely aligned in the representation space.

After the user's question is converted into an embedding, the application compares that query embedding with every question embedding in the knowledge base. Scikit-learn's `cosine_similarity` function performs this calculation. The resulting array contains one similarity score for each stored question.

The best candidate is selected using NumPy's `argmax` function. The index returned by `argmax` is used to locate the corresponding question and answer in the Pandas DataFrame. This keeps the retrieval process simple and transparent.

Cosine similarity is appropriate here because the Sentence Transformer produces dense semantic vectors. The method focuses on the orientation of vectors rather than their raw magnitude. The project normalizes embeddings before comparison, making the similarity calculation particularly convenient. The retrieval result includes the best matching question, its score, and the associated answer. These values are also used by the user interface to display retrieval details and to decide whether the threshold has been satisfied.

```
similarity_scores = cosine_similarity(
    [query_embedding], question_embeddings
)[0]
best_index = int(np.argmax(similarity_scores))
best_score = float(similarity_scores[best_index])
```

12. Similarity Threshold and Fallback

A retrieval system should not blindly return the closest record because the closest record may still be unrelated to the user's question. EduBot addresses this problem using a configurable similarity threshold. The default threshold in the project is 0.55, and the Streamlit sidebar provides a slider that allows the user to adjust it between 0.30 and 0.90.

After cosine similarity is calculated, the system identifies the highest score. If that score is greater than or equal to the selected threshold, the corresponding knowledge-based answer is returned. If the score is below the threshold, the chatbot displays a controlled fallback response explaining that it could not find a relevant answer in the educational knowledge base.

The threshold therefore acts as a simple acceptance rule. A lower threshold increases the chance of returning an answer but can also allow weaker matches. A higher threshold makes the system more conservative but may increase fallback responses. The appropriate value depends on the dataset and desired behavior.

The fallback message also guides users toward supported topics of **(Data Science)** such as AI, Machine Learning, Deep Learning, Python, Data Science, Statistics, NLP, Computer Vision, SQL, and Generative AI. This improves the user experience by explaining what kinds of questions the system is intended to answer.

```
If best_score >= threshold:
    answer = best_answer
else:
    answer = "Sorry, I couldn't find a relevant answer..."
```

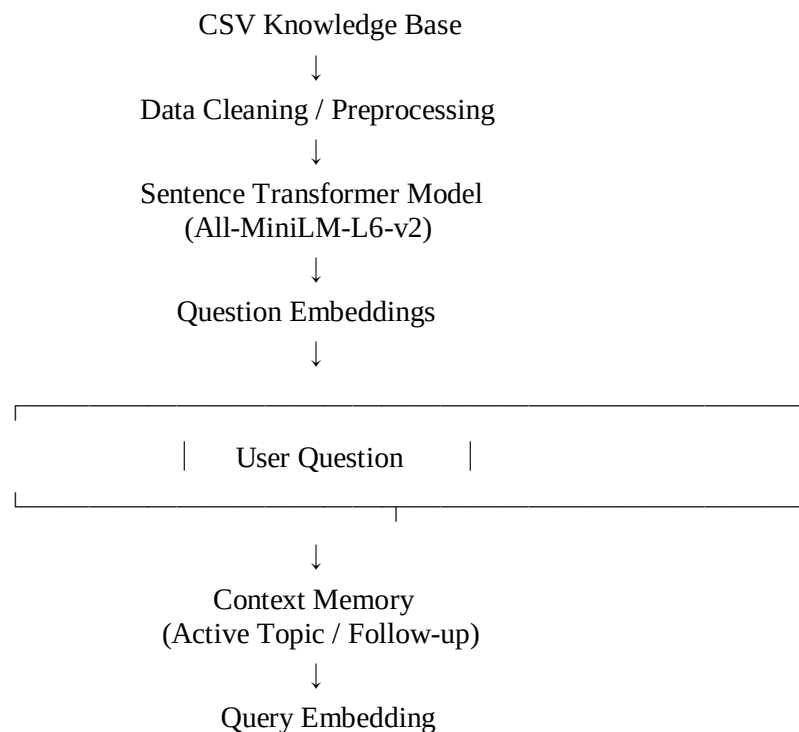
13. Context-Aware Follow-Up Question

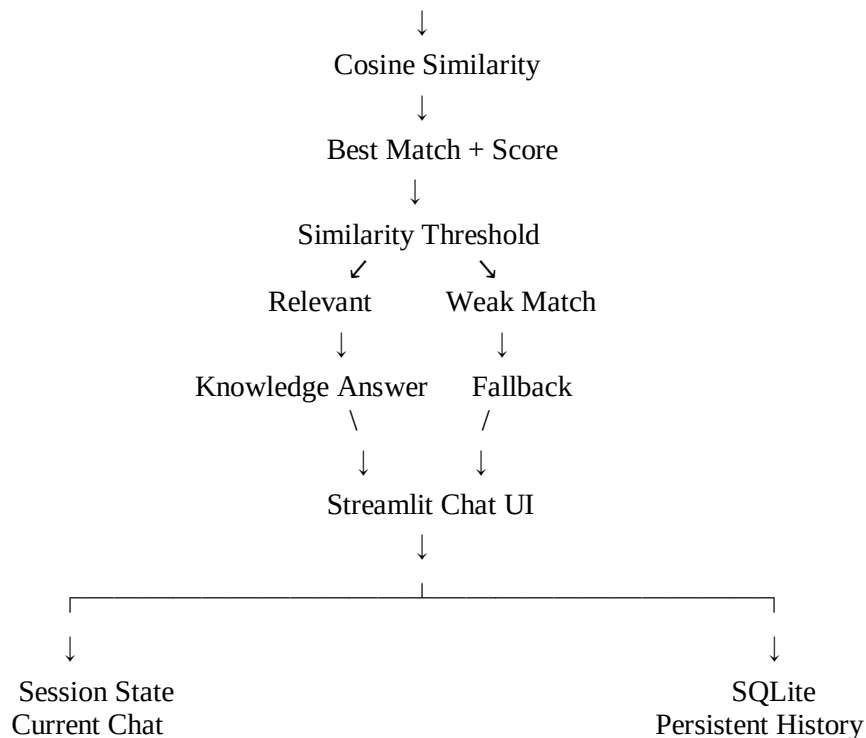
The context memory component in EduBot AI is responsible for understanding the relationship between the user's current question and the questions asked earlier in the conversation. Instead of treating every question as completely independent, the chatbot maintains an active topic/context from the previous conversation and uses it to interpret follow-up questions. For example, if the user first asks "What is Python?" the chatbot identifies Python as the active topic. If the user then asks "What is its type?" this question by itself is ambiguous because the word "*its*" does not specify what it refers to. The context memory connects it with the previous topic and internally interprets the question as something like "What are the types of Python data types?" allowing the semantic-search system to retrieve a more relevant answer. Similarly, if the user changes the topic by asking "What is SQL?" the chatbot updates the active topic from Python to SQL. If the next question is "What is its type?" the chatbot now understands that "*its*" refers to SQL and searches for information related to SQL data

types. This mechanism combines conversation history, active-topic detection, follow-up-question detection, and semantic search. Therefore, context memory does not simply store previous messages; it uses previous conversation information to resolve ambiguous follow-up questions, maintain continuity, and switch context when the user starts a genuinely new topic. This makes EduBot AI more conversational and closer to a real educational assistant rather than a simple question-answer retrieval system.

14. System Architecture

The EduBot AI system architecture is designed as a layered pipeline that connects the educational knowledge base, NLP-based semantic retrieval, application logic, user interface, and persistent storage. The system begins with the CSV knowledge base, which contains structured educational questions and answers. The data is cleaned by removing unnecessary spaces, duplicate records, and empty values. The cleaned questions are then passed to the all-MiniLM-L6-v2 Sentence Transformer, which converts each question into a numerical semantic embedding. When the user enters a question through the Streamlit chat interface, the same model converts the user's query into an embedding. The system then uses cosine similarity to compare the query embedding with all stored question embeddings and identifies the question with the highest similarity score. The context-memory layer helps interpret follow-up questions by using the active conversation context—for example, after “What is Python?” a follow-up such as “What is its type?” can be interpreted in the Python context rather than as an unrelated question. If the user explicitly introduces a new topic, the active context can be switched to that topic. After retrieval, the similarity-threshold layer checks whether the best match is sufficiently relevant. If the score satisfies the threshold, the corresponding answer from the knowledge base is returned; otherwise, the chatbot gives a controlled fallback response. Finally, the conversation is maintained in Streamlit session state for the active chat and stored in SQLite so that chat history can remain available after refreshes or application restarts. Thus, the updated architecture combines semantic search + context-aware conversation + threshold-based fallback + Streamlit UI + session memory + SQLite persistence into one educational chatbot system.





15. Streamlit User Interface

Streamlit provides the interactive front end for EduBot AI. The application is designed to make the retrieval system easy for a student to use without requiring knowledge of Python or the underlying NLP implementation. The main page displays the EduBot AI title, a short description, the conversation, and a chat input.

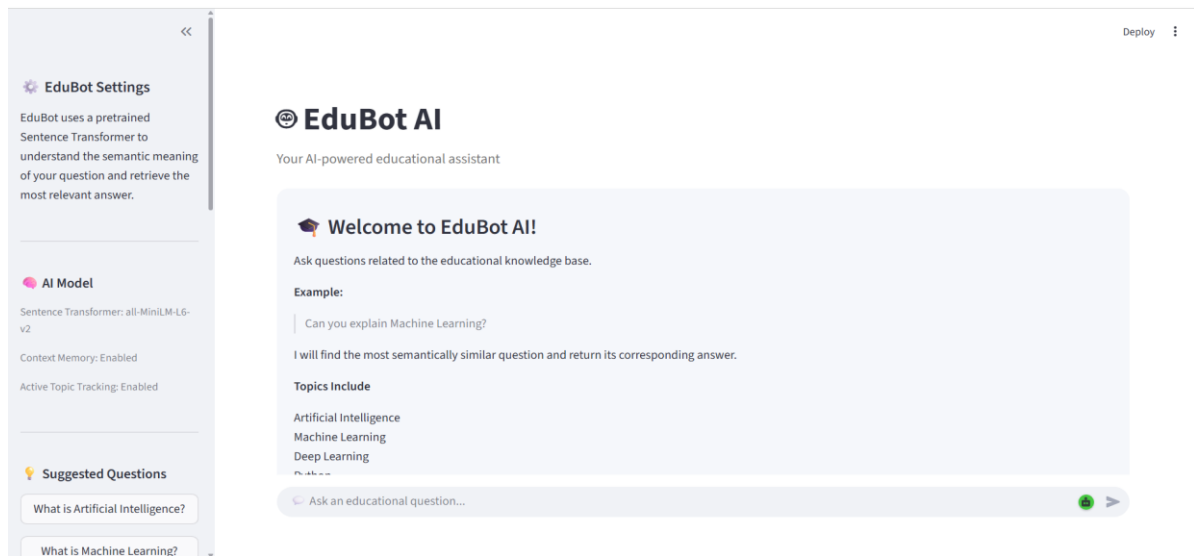
The sidebar provides system information and controls. It displays the number of questions in the knowledge base and identifies the Sentence Transformer model. Suggested questions are presented as buttons so that users can test the chatbot quickly. A similarity-threshold slider allows the user to control how strict the retrieval system should be.

When an answer is returned, the interface shows the semantic similarity score. The retrieval-details expander provides the best matching question, the similarity score, and the active threshold. This makes the system more transparent because the user can see why a particular answer was selected.

The application also contains a clickable Chat History control. The history opens in a Streamlit popover and displays previously stored conversations in a table. A clear-history button removes stored records from SQLite and resets the current session.

The interface is intentionally lightweight. Streamlit handles widgets, layout, state, and rendering while the Python retrieval functions perform the NLP operations.

[SCREENSHOT PLACEHOLDER: Main EduBot AI Interface]



16. Session-Based Chat History

Streamlit session state is used to maintain the visible conversation while the application is running. The project stores messages in `st.session_state.messages`. Each user message is stored with the role “user” and its text, while each assistant response stores the answer along with retrieval information such as the similarity score and match status.

When the application reruns after an interaction, the existing session state remains available, allowing previous messages to be displayed again. The application loops through the messages and uses `st.chat_message` to render the appropriate user or assistant message.

Session state is useful for the active conversation, but it is not intended to be the permanent database. If the session disappears, the in-memory message list can also disappear. For that reason, EduBot combines session state with SQLite. At application initialization, records from SQLite are loaded into session state so that previous conversations can be displayed in the chat interface.

This dual approach gives the application both a responsive conversation experience and persistent storage. Session state controls the current UI, while SQLite provides durable history.

If “messages” not in `st.session_state`:

```
st.session_state.messages = []
```

```
st.session_state.messages.append({
    "role": "user",
    "content": user_question
})
```

17. SQLite Persistent Chat History

SQLite provides local persistent storage for EduBot AI's conversation history. Unlike session state, which primarily maintains state during an active Streamlit session, SQLite stores records in a database file named `chat_history.db`. This means conversations can remain available after a browser refresh or application restart.

The project creates a chat_history table containing an automatically generated id, the question, answer, similarity score, matched question, match status, and timestamp. Each time the chatbot processes a user question, the question and generated answer are inserted into the table along with retrieval information.

The database is accessed through Python's built-in sqlite3 module. Panda's read_sql_query is used to retrieve history into a DataFrame for display. The Streamlit Chat History popover then shows selected fields in a table.

The persistent design is simple and appropriate for a local educational prototype. A larger multi-user production system would require stronger database architecture, authentication, authorization, backups, and concurrency planning.

```
CREATE TABLE IF NOT EXISTS chat_history (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  question TEXT NOT NULL,
  answer TEXT NOT NULL,
  similarity_score REAL,
  matched_question TEXT,
  matched INTEGER,
  timestamp TEXT NOT NULL
);
```

Field	Description
id	Unique conversation record ID
question	User's submitted question
answer	Returned chatbot answer
similarity_score	Best cosine similarity score
matched_question	Question selected from the CSV
matched	Whether the threshold was satisfied
timestamp	Date and time of interaction

[SCREENSHOT PLACEHOLDER: SQLite Chat History]

	A	B	C	D	E
1	ID	Question	Answer	Similarity	Date & Time
2	2	What is NLP?	Natural Language Processing (NLP) is the a	100.00%	2026-08-19 13:09:52
3	1	What is Compute	Computer vision is an AI field that enables c	100.00%	2026-08-19 13:09:46

18. Complete System Workflow

The complete EduBot workflow begins when the application is launched. First, the CSV knowledge base is loaded and cleaned. The application selects the question and answer columns, converts them to strings, removes unnecessary whitespace, eliminates duplicate pairs, and filters empty records.

Next, the all-MiniLM-L6-v2 Sentence Transformer is loaded. The cleaned questions are encoded into semantic embeddings. Streamlit caching allows the model and embeddings to be reused rather than recreated unnecessarily on every application rerun.

When a user enters a question, the chatbot removes unnecessary leading and trailing spaces. The query is passed to the Sentence Transformer and converted into an embedding. If it is a contextual follow-up, the active topic is used to form a more meaningful retrieval query. Cosine similarity is calculated between the query vector and all stored question vectors. The system identifies the index with the highest similarity score.

The selected score is compared with the threshold chosen in the sidebar. If the score meets the threshold, the corresponding answer is returned. Otherwise, the fallback response is shown. The result is displayed along with similarity information and an optional retrieval-details section.

The user question and chatbot answer are then added to session state for the current conversation and saved to SQLite for persistence. When the user clicks Chat History, the application reads the database and presents the previous records in a table.

User → Chat Input → Query Embedding → Cosine Similarity

→ Best Matching Question → Threshold Check

→ Answer / Fallback → Streamlit UI → SQLite History

19. Testing and Validation

Testing is required to confirm that the chatbot behaves correctly for normal questions, unrelated questions, empty input, persistence, and history management. EduBot can be validated using functional test cases that cover the major components of the system.

A relevant question such as “What is Artificial Intelligence?” should produce a matching answer when the similarity score meets the threshold. A differently worded question with the same meaning can be used to verify semantic retrieval. An unrelated question should produce the fallback response when its best score is below the threshold. The chatbot can retain the current conversational topic so that short follow-up questions can be interpreted meaningfully. For a genuinely new subject, the system can switch the active topic and retrieve from the new context. Empty input should not trigger an unnecessary retrieval operation.

The Streamlit interface should also be tested. Suggested-question buttons should populate and process the selected query. The similarity slider should change the acceptance condition. Retrieval details should show the selected question and score. The Chat History popover should display stored records in table format.

Persistence testing is particularly important. After asking several questions, the application can be refreshed or restarted. The previously stored records should still be available because they are stored in SQLite. The clear-history button should remove records from both the database and the active session.

Test ID	Test Input / Action	Expected Result
TC01	What is AI?	Relevant answer is returned
TC02	Explain Machine Learning	Semantic match is found
TC03	Unrelated question	Fallback response
TC04	What is Python? What is its type?	Follow-up context
TC05	Empty input	Validation response
TC06	Refresh browser	Stored history remains
TC07	Restart app	SQLite history remains
TC08	Click Chat History	History table opens
TC09	Clear Chat History	History is deleted

20. Results and Discussion

The completed EduBot AI prototype demonstrates that a compact retrieval-based NLP architecture can provide an effective educational question-answering experience when supported by a curated

knowledge base. The Sentence Transformer enables semantic comparison between a user query and stored questions, allowing the system to handle wording variations better than exact string matching. The cosine similarity score provides an interpretable retrieval signal. The user can see the score in the interface and inspect the best matching question through the retrieval-details section. The threshold mechanism adds a simple form of confidence control: the system returns an answer when the similarity is sufficiently high and otherwise uses the fallback response.

The context-aware follow-up mechanism improves conversation quality for short questions. It allows the chatbot to retain the active subject while still switching to a new topic when the user clearly introduces one.

The Streamlit interface makes the system practical to demonstrate. Users can type questions, choose suggested questions, adjust the threshold, inspect retrieval details, and view previous conversations. The sidebar also communicates the size of the knowledge base and the model being used.

SQLite adds an important persistence capability. Conversations are no longer limited to the active session because question-answer records are stored in `chat_history.db`. The clickable Chat History popover provides a compact way to inspect previous records, including the question, answer, similarity score, and timestamp.

Results should be evaluated primarily through test cases rather than claiming a universal accuracy percentage unless a formal benchmark has been performed. For a retrieval-based chatbot, dataset coverage, semantic similarity quality, threshold selection, and answer quality all influence observed behavior.

21. Advantages

EduBot AI has several advantages as a project prototype. First, its architecture is relatively simple and easy to understand. The complete pipeline—from CSV loading to semantic retrieval, interface rendering, and SQLite storage—can be implemented in a single Python application.

The use of Sentence Transformers provides semantic retrieval rather than depending only on exact keyword matches. This improves the system's ability to recognize questions with similar meanings but different wording. The all-MiniLM-L6-v2 model provides a practical embedding approach for a small educational application.

Another advantage is the structured knowledge base. Because answers are stored in a CSV file, the system is predictable: it retrieves an answer from the curated dataset rather than generating unrestricted information. Updating the knowledge base can be as simple as adding or editing question-answer records.

The similarity threshold provides a transparent mechanism for rejecting weak matches. Users can adjust the threshold through the Streamlit sidebar, making the behavior easy to demonstrate.

Context-aware follow-ups make short conversational questions more meaningful.

The Streamlit interface is another advantage. It provides a browser-based application without requiring a separate front-end framework. Suggested questions, retrieval details, chat input, and history controls make the prototype interactive.

SQLite adds persistent storage without requiring a separate database server. This is convenient for local development and demonstrations. Finally, the project does not require an external generative-AI API to answer its core questions, which reduces operational complexity for the prototype.

22. Limitations

The most important limitation of EduBot AI is that it is dependent on its knowledge base. The chatbot can reliably answer only questions that are represented adequately by the CSV dataset. If a user asks about a topic that is absent from the knowledge base, the system cannot create a complete new answer

from external knowledge. Instead, it relies on the similarity threshold to decide whether to return the fallback response.

The system is also retrieval-based rather than generative. It returns the stored answer associated with the best matching question. Therefore, it does not synthesize multiple records into a new explanation or carry out open-ended reasoning in the manner of a large language model.

The similarity threshold requires tuning. A threshold that is too low may allow weak matches and produce less relevant answers. A threshold that is too high may reject useful matches and produce too many fallback responses. The optimal value depends on the dataset and desired balance between coverage and precision.

Another limitation is dataset quality. If an answer is incomplete, unclear, outdated, or incorrect, the chatbot will reproduce that problem when the corresponding question is selected. Semantic retrieval can also struggle with ambiguous questions or questions that have several possible interpretations. SQLite is suitable for a local prototype and small-scale application, but a larger multi-user system would require stronger database design, concurrency considerations, authentication, access control, and cloud infrastructure.

23. Future Scope

EduBot AI can be extended significantly beyond the current CSV-based retrieval architecture. The first improvement would be to expand the knowledge base and introduce more diverse question formulations. A larger and better-labeled dataset would improve coverage and provide stronger evaluation opportunities.

A document-based retrieval system could also be added. Instead of limiting knowledge to a CSV, the application could process PDFs, notes, textbooks, or other educational documents. This could lead to a Retrieval-Augmented Generation architecture in which relevant document chunks are retrieved and then passed to a language model for natural-language synthesis.

Hybrid search is another possible improvement. Semantic embeddings could be combined with lexical methods such as TF-IDF or keyword search. This could improve retrieval for exact technical terms while preserving semantic matching.

Multilingual support could make the chatbot more useful to a wider student population. Voice input and text-to-speech could provide an accessible conversational experience.

For larger deployments, user authentication and user-specific history should be added. A cloud database could replace local SQLite, and administrative tools could allow authorized users to manage the knowledge base. Analytics could track common questions, fallback rates, similarity-score distributions, and user feedback.

The retrieval system could also be evaluated systematically using a labeled test dataset. Metrics such as top-1 accuracy, recall, precision, and accepted-match precision could be monitored.

Finally, the application could be deployed to a cloud environment and integrated with monitoring, logging, automated testing, and version control. These improvements would transform the current educational prototype into a more scalable intelligent learning assistant.

23. Conclusion

EduBot AI demonstrates the development of a practical educational question-answering chatbot using semantic retrieval. The project begins with a structured CSV knowledge base containing approximately question-answer pairs and applies lightweight text cleaning to prepare the content for semantic encoding. The cleaned questions are transformed into embeddings using the all-MiniLM-L6-v2 Sentence Transformer.

The central retrieval mechanism is cosine similarity. A user's question is encoded into a vector and compared with the stored question embeddings. The highest-scoring candidate is selected, and a configurable similarity threshold determines whether its answer is relevant enough to return. This threshold also supports a controlled fallback response for unrelated or weakly matched questions.

The Streamlit application provides an accessible interface through which users can ask questions, use suggested questions, adjust the similarity threshold, inspect retrieval details, and manage their

conversation. Session state maintains the active conversation, while SQLite provides persistent storage for question-answer interactions.

The persistent history feature demonstrates the difference between temporary application state and durable storage. Records stored in `chat_history.db` remain available after refreshing or restarting the application. The clickable Chat History popover presents these records in a table, while the clear-history feature allows the stored conversation data to be removed.

Overall, the project meets the goal of creating a lightweight, understandable, and demonstrable AI/ML educational chatbot without requiring an external generative-AI API. Its modular concepts—clean data, semantic embeddings, similarity retrieval, threshold-based fallback, interactive UI, and persistent history—provide a strong foundation for future development.

24. References

The project uses software libraries and technologies whose official documentation should be consulted when maintaining or extending the application. References should be limited to resources actually used during development. The following technologies are directly relevant to EduBot AI: Python for programming and the standard library, Pandas for CSV processing, NumPy for numerical operations, Sentence Transformers for semantic embeddings, scikit-learn for cosine similarity, Streamlit for the web interface, and SQLite for persistent local storage.

When submitting the final report, the exact URLs and access dates can be added according to the citation style required by the institution. Screenshots of the actual application and dataset should be labeled as project artifacts rather than treated as external references.

Suggested references include the official Python documentation, Pandas documentation, NumPy documentation, Sentence Transformers documentation, scikit-learn documentation, Streamlit documentation, and SQLite documentation.

Technology	Reference
Python	Official Documentation
Pandas	Official Documentation
NumPy	Official Documentation
Sentence Transformers	Official Documentation
Scikit-learn	Official Documentation
Streamlit	Official Documentation
SQLite	Official Documentation